

Experiment - 1

*> Aim :- To implement Boston Housing Price Prediction using linear Regression with a Deep Neural Network

* Objective

·> To understand regression using deep learning

·> To predict continuous values using neural networks.

·> To analyze performance using loss functions

·> To learn dataset preprocessing and normalization.

* System Requirements

·> Python 3.x

·> Libraries : Numpy, Pandas, Matplotlib, TensorFlow

·> Jupyter Notebook

* Theory

·> Linear regression is a supervised learning technique used to predict continuous output using a linear equation.

- > In deep learning, linear regression can be implemented using a neural network with no hidden layers
- > A Deep Neural Network (DNN) consists of input layer, hidden layers, and output layer where weights and biases are adjusted using backpropagation.
- > Boston housing Dataset contains features like crime rate, number of rooms, and property tax.
- > Each neuron computes a weighted sum of inputs followed by an activation function such as ReLU.
- > The output layer uses a linear activation function since the output is continuous
- > Loss function used is Mean Squared Error (MSE), which calculates the average squared difference between predicted and actual values.
- > Gradient Descent is used to minimize loss by updating weights iteratively

* Algorithm

- 1> Load Boston housing dataset
- 2> Split dataset into training and testing
- 3> Normalize input features
- 4> create neural network model
- 5> Add input, hidden and output layers
- 6> compile model using optimizer and loss function
- 7> Train model on training data
- 8> Evaluate model on test data
- 9> Predict house prices
- 10> Display results.

Example

Consider a small dataset:

No. of rooms	Prices (in Lakhs)
2	30
3	45
4	60
5	80

Here, input = no. of rooms, output = house price

→ The model learn relationship like

$$\text{Price} = w \times \text{Rooms} + b$$

Explanation

→ Suppose the neural network learns!

$$w = 15$$

$$b = 0$$

Then prediction

$$\text{For 3 rooms} \rightarrow \text{price} = 15 \times 3 = 45$$

$$\text{For 5 rooms} \rightarrow \text{Price} = 15 \times 3 = 75$$

→ In real Boston dataset!

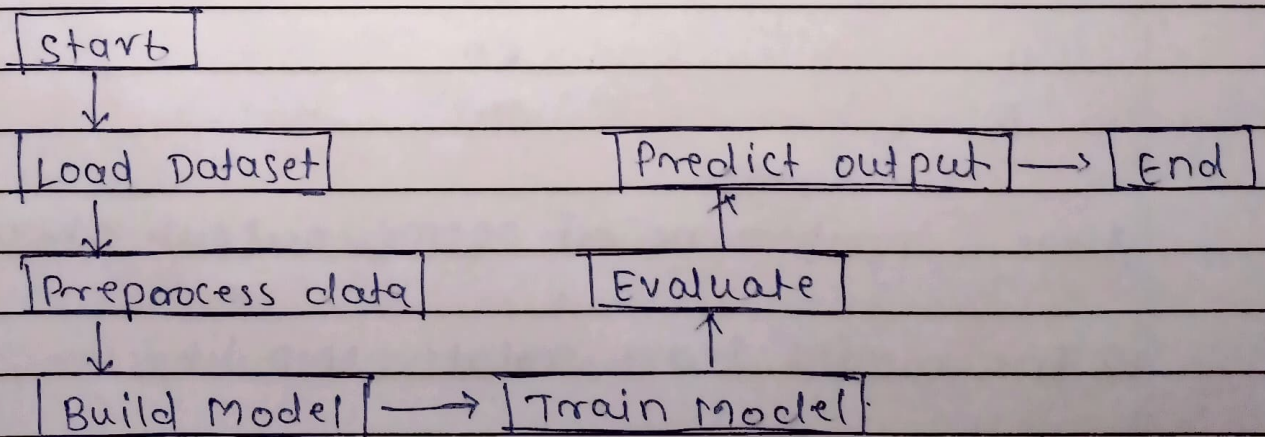
Inputs = crime rate, no. of room, location

output = house price

→ If number of rooms increases and crime rate decreases $\rightarrow$ price increases

→ model learns this pattern automatically

* Flowchart



* Result

→ The model successfully predicts house prices with minimum error. Accuracy improves with proper tuning of layers and learning rate.

* Conclusion

→ Deep Neural Networks can effectively perform linear regression tasks and improve prediction accuracy compared to traditional methods when handling complex datasets.

Experiment - 2

* **Aim :-** To classify movie reviews as positive or negative using Deep Neural Network.

* **Objective**

- > To understand binary classification
- > To process text data for deep learning
- > To build sentiment analysis model
- > To evaluate classification accuracy
- > To learn embedding and vectorization.

* **System Requirement**

- > Python 3.x
- > Libraries : TensorFlow / Keras, Numpy, Pandas
- > Jupyter Notebook
- > Minimum 4GB RAM

* **Theory**

- > Classification is a supervised learning technique where output is categorical such as positive or negative sentiment.

- 1. Binary classification involves only two classes and is widely used in sentiment analysis applications.
- 2. Deep Neural networks can learn complex patterns from text data using embeddings and dense layers.
- 3. IMDB dataset contains labeled movie reviews used for sentiment classification tasks.
- 4. Text data must be converted into numerical format using tokenization and padding techniques.
- 5. Embedding layer converts words into dense vectors capturing semantic meaning.
- 6. Activation functions like sigmoid is used in output layer to produce probability between 0 and 1.
- 7. Binary Cross-Entropy loss is used to measure classification error.

→ Backpropagation adjusts weights to minimize classification error.

→ Example: Review "This movie is amazing" is classified as Positive, while "worst film ever" is classified as negative

* Algorithm

- 1> Load IMDB dataset
- 2> Preprocess text (tokenization, Padding)
- 3> Split dataset into train / test
- 4> Build dataset neural network model
- 5> Add embedding layer
- 6> Add hidden dense layers
- ~~7> Add hidden dense layers~~
- 7> Add output layer with sigmoid
- 8> compile model
- 9> Train model
- 10> Evaluate and predict

Example

consider movie reviews :

Review Text	Output
"This movie is amazing"	Positive (1)
"worst movie I have seen"	Negative (0)

Step 1: Text to Numbers

"amazing" $\rightarrow$ 245

"worst" $\rightarrow$ 678

$\rightarrow$ words are converted into numbers using tokenization

Step 2: Neural Network Prediction

$\rightarrow$ Model processes the sequence of numbers

$\rightarrow$ Outputs probability using sigmoid

ex. output

$\rightarrow$ Review = "This movie is amazing"

$\rightarrow$ output = 0.92 $\rightarrow$ Positive

$\rightarrow$ Review = "worst movie ever"

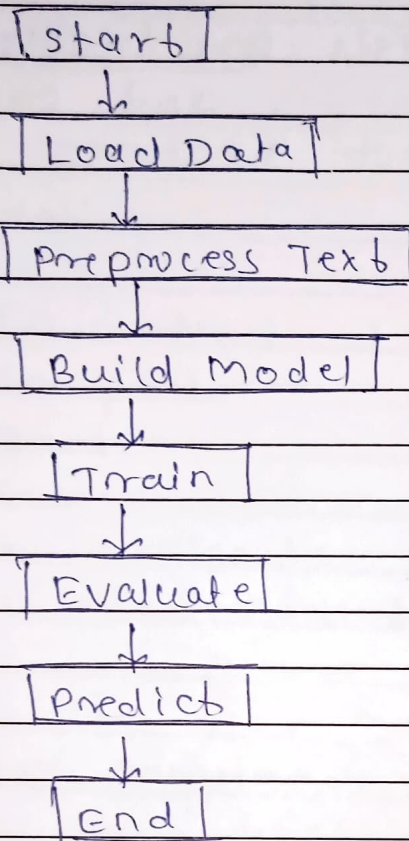
$\rightarrow$ output = 0.10 $\rightarrow$ Negative

Explanation

$\rightarrow$ If output $> 0.5 \rightarrow$ positive

$\rightarrow$ If output $< 0.5 \rightarrow$ Negative

* Flowchart



* Result

→ The model correctly classifies movie reviews with good accuracy (~85-90 % depending on tuning)

→ Conclusion

→ Deep learning models are highly effective for sentiment analysis and can automatically extract features from text data.

Experiment - 3

* Aim :- To design a CNN model to classify fashion items using fashion MNIST dataset.

* Objective

- > To understand CNN architecture.
- > To perform image classification
- > To learn feature extraction using convolution
- > To evaluate accuracy of model
- > To understand pooling and filters.

* System Requirements

- > Python 3.x
- > Libraries : TensorFlow / Keras, Numpy, Matplotlib
- > Jupyter Notebook
- > GPU & optional RAM

* Theory

- > CNN is a deep learning model mainly used for image processing and classification tasks.

- 1) It uses convolution layers to extract features such as edges, textures, and patterns from images.
- 2) Filters (kernels) slide over image to produce feature maps capturing important information.
- 3) Pooling layers reduce image size and computation while retaining important features.
- 4) ReLU activation introduces non-linearity to the model.
- 5) Fully connected layers are used for final classification.
- 6) Fashion MNIST dataset contains images of clothing such as shirts, shoes, and bags.
- 7) Softmax activation is used in output layer for multi-class classification.
- 8) CNN reduces manual feature extraction compared to traditional image processing methods.

* Algorithm

- 1> Load fashion MNIST dataset
- 2> Normalize pixel values
- 3> Reshape data for CNN
- 4> Build CNN model
- 5> Add convolution + pooling layers
- 6> Add Flatten and dense layers
- 7> compile model
- 8> Train model
- 9> Evaluate accuracy
- 10> Predict output

Example

consider dataset contains images like !

·> T-shirt

·> shoe

·> Bag

Each image is 28×28 pixels

·> Example input image:

Image of a shoe (pixel values)

Step 1: Convolution

- > Filter detects edges of shoe
- > Another filter detects shape.

Step 2 : Pooling

- > Reduces image size
- > keeps important features

Step 3 : classification

- > Final output :

class	Probability
T-shirt	0.05
Shoe	0.90
Bag	0.05

predicted class = shoe

Explanation

- > CNN automatically detects :

- > Edges
- > Patterns
- > Shapes

- > Unlike normal neural networks :

- > No need to manually extract features

Example :

- > shoe → curved edges, sole pattern
- > shirt → Flat structure

→ Model learns these differences during training

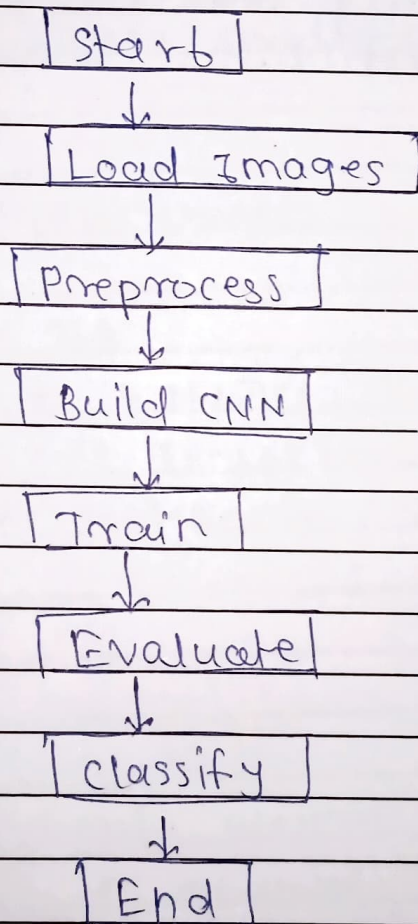
→ Real-world use:

→ Face recognition

→ Medical image analysis

→ Self-driving cars.

* Flowchart



* Result

→ CNN achieves high accuracy ($\sim 90\%+$) in classifying fashion images correctly.

* Conclusion

→ CNN achieves is highly effective for image classification tasks and provides better accuracy due to automatic feature extraction.

Experiment-4

* **Aim** :- To design a time series prediction system using Recurrent Neural Network for google stock price dataset.

* **Objective**

- To understand time series data
- To implement RNN for sequential data
- To predict future stock prices
- To analyze trends and patterns
- To evaluate prediction accuracy

* **System Requirements**

- Python 3.x
- Libraries :- Numpy, Pandas, Matplotlib, TensorFlow / Keras
- Jupyter Notebook / VS Code

* **Theory**

- Recurrent Neural Network (RNN) is a type of neural network designed to handle sequential or time-dependent data such as stock prices, text, and speech.

- > Unlike traditional neural networks, RNN has memory which stores previous information and uses it for future predictions.
- > In stock prediction, past prices influence future prices, making RNN suitable for time series analysis.
- > RNN processes input in sequence form, where each step depends on the previous step's output.
- > Hidden states in RNN carry information from previous time steps and help in learning temporal dependencies.
- > ~~Hidden~~ Long Short-Term memory (LSTM) is an advanced type of RNN that overcomes vanishing gradient problem and remembers long-term dependencies.
- > Input data is usually normalized to improve training performance and stability.

* Input data is usually normalized to improve training performance and

→ The model predicts next value based on previous sequence of values (window-based prediction).

→ Loss function such as Mean Squared Error (MSE) is used to calculate prediction error.

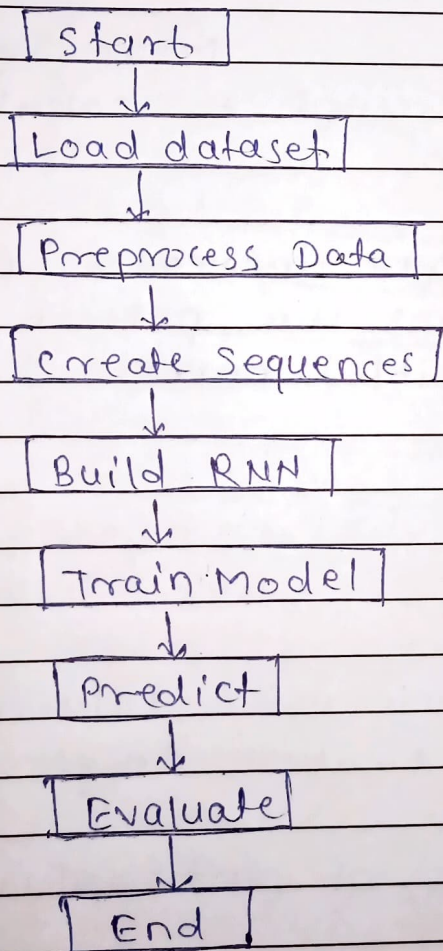
→ Ex, if stock price increases for last 5 days, RNN learns this pattern and predicts a possible increase on next day.

* Algorithm

- 1) Load Google stock price dataset
- 2) select relevant feature (e.g closing price)
- 3) Normalize data
- 4) create sequences of past value (time steps)
- 5) Split into training and testing data
- 6) Build RNN / LSTM model

- 7> Add layers and compile model
- 8> Train model on training data
- 9> Predict stock prices
- 10> Evaluate performance

* Flowchart



ay)

Example

Sample Dataset

Day	Price
-----	-------

1	100
---	-----

2	102
---	-----

3	105
---	-----

4	107
---	-----

5	110
---	-----

Step 1: Create Sequence

→ Input (last 3 days) → Output (next day)

example: → [100, 102, 105] → 107

→ [102, 105, 107] → 110

Step 2: Model Learning

→ RNN learns pattern:

→ prices are increasing gradually

Step 3: - Prediction

→ Input: [105, 107, 110]

→ Predicted output: 112

Explanation

- > RNN uses previous values to predict next value
- > It remembers trends like increasing or decreasing prices.
- > If trend is upward $\rightarrow$ prediction increases
- > If trend is downward $\rightarrow$ prediction decreases

* Result

- > The RNN/LSTM model successfully predicts stock price trends with reasonable accuracy and captures time-dependent patterns effectively.

* Conclusion

- > RNN is highly suitable for time series prediction tasks such as stock price forecasting, as it utilizes past sequential information to predict future values.